4.2. Python и ИБ API и анализ данных

4. Python в задачах информационной безопасности

В этом уроке

- Узнаем, как применяются API в информационной безопасности
- Познакомимся с анализом данных и инструментами Python
- Изучим применение статистических методов в ИБ

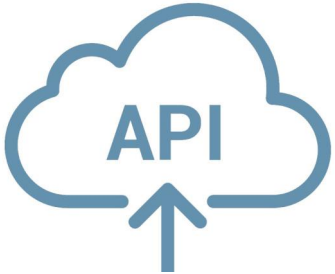
Что такое API

API (Application Programming Interface) — интерфейс программирования приложений, который позволяет одной программе взаимодействовать с другой.

Работает как «переводчик»: вы отправляете запрос в определённом формате — получаете ответ, часто в формате JSON.

Примеры API-запросов в ИБ:

- Сканер на рабочем месте отправляет hash файла в VirusTotal → получает список антивирусных движков и их мнение.
- SIEM-система запрашивает у Shodan данные об открытых портах внешнего IP-адреса



Зачем использовать API в безопасности?

- **Автоматизация**: API позволяют быстро собирать информацию без ручного поиска. Это критично при реагировании на инциденты.
- **Интеграция**: различные инструменты безопасности (антивирусы, SIEM, сканеры уязвимостей) могут «общаться» между собой.
- **Скорость**: вместо открытия 5 сайтов вручную — 1 запрос в API и мгновенный результат.
- **Масштабируемость**: можно запускать массовые проверки сотен IP/URL автоматически.

Важные API, которые стоит знать

Таблица или маркированный список:

- VirusTotal API — анализ файлов и URL по базам AV-движков
- Shodan API — поиск устройств и сервисов, подключённых к интернету
- SecurityTrails API — DNS, WHOIS, домены, история IP
- Vulners API — база уязвимостей и патчей (CVE, ExploitDB и др.)
- AbuseIPDB API — база подозрительных IP-адресов (боты, спамеры)

VirusTotal API

VirusTotal позволяет делать массовый анализ подозрительных объектов

- Включает сервис Google/Chronicle для анализа файлов и URL более чем 70 движками.
- Отправку файлов, хэшей, URL → JSON-ответ с:
  - результатами сканирования,
  - уровнями угрозы,
  - ссылками на отчёты,
  - тегами типа «malware», «phishing».

Есть бесплатные и платные тарифы, ограничение на частоту запросов.

Shodan API

Shodan — поисковик уязвимых устройств

Позволяет искать по:

- IP, доменам, портам, сервисам
- метаданным баннеров (Apache, nginx, RDP и др.)

Используется для:

- выявления открытых портов на периметре компании,
- сканирования партнёров или провайдеров,
- анализа атакующих (если IP попал в инцидент).

API внутри инфраструктуры

- OpenDXL (McAfee) — шина для обмена данными между продуктами безопасности.
- Cuckoo Sandbox API — запускает анализ файлов в песочнице, возвращает отчёты (поведение, сетевые соединения).
- TheHive API — платформа для управления инцидентами. Можно автоматически создавать кейсы, загружать индикаторы, интегрироваться с Cortex.

API — это не только внешний доступ, но и внутренняя интеграция.

Например: SIEM → API → TheHive → Cuckoo → анализ → кейс.

Что даёт использование API в ИБ?

API = автоматизация + интеграция + эффективность

Использование API:

- ускоряет анализ инцидентов;
- повышает качество расследования;
- облегчает рутину аналитика.

Без API невозможно построить современную инфраструктуру ИБ (SIEM, SOAR, SOC, EDR).

API — ключ к масштабируемой кибербезопасности

Зачем анализировать данные в ИБ

Цели анализа данных в информационной безопасности:

- Распознавание угроз
  - Пример: Выявление подозрительных IP-адресов в логах.
- Прогнозирование атак
  - Пример: Анализ временных рядов для предсказания DDoS.
- Выявление уязвимых мест
  - Пример: Частые ошибки аутентификации в определенных сервисах.

Этапы обработки данных

Удаление шума и дубликатов:

В логах встречаются повторяющиеся события (например, дубликаты запросов из-за ошибок логирования). «Шум» — это нерелевантные данные (например, служебные записи)

```
import pandas as pd

# Удаление дубликатов
df = df.drop_duplicates()

# Удаление строк с шумом (например, тестовые данные)
df = df[~df['action'].str.contains('test', na=False)]
```

Этапы обработки данных

Заполнение пропусков:

- Проблема: В данных есть NaN или null (например, отсутствует время события).
- Методы:
  - Замена средним/медианой: Для числовых данных (например, время ответа сервера).
  - Интерполяция: Если данные временные.

```
# Заполнение пропусков в столбце 'response_time' медианой
df['response_time'] = df['response_time'].fillna(df['response_time'].median())

# Замена пропусков в категориальных данных
df['user_agent'] = df['user_agent'].fillna('unknown')
```

Этапы обработки данных

Преобразование в таблицы:

Проблема: Данные могут быть в формате JSON, лог-файлов или даже сырого текста.

Решение:

- CSV → DataFrame: Стандартный импорт.
- JSON → DataFrame: Нормализация вложенных структур.
- Логи → DataFrame: Парсинг с помощью регулярных выражений.

```
# CSV в DataFrame
df = pd.read_csv('security_logs.csv')

# JSON в DataFrame (если данные вложенные)
df = pd.json_normalize(json_data, record_path=['events'])
```

Финальные выводы

Тезисы:

- API + Python = автоматизация рутинных задач.
- Очистка данных повышает точность детектирования.
- Графики помогают убедить руководство в рисках.

Итоги урока

- Узнали, как применяются API в информационной безопасности
- Познакомились с анализом данных и инструментами Python
- Изучили применение статистических методов в ИБ